## CS F364: Design & Analysis of Algorithms

**Quiz 4 — June 11, 2026**

*Coverage: Lectures 1–14*

> **Note**
>
> **Instructions:**
>
> - Time: 15 minutes.
> - Write your answers clearly on paper, scan, and upload to Google Classroom.
> - Show all steps for full credit.

---

## Question: Huffman Coding

A file contains characters with the following frequencies:

| Character | A | B | C | D | E |
|---|---|---|---|---|---|
| **Frequency** | 10 | 15 | 30 | 5 | 40 |

We want to design an optimal prefix code for these characters using Huffman's greedy algorithm.

**(a) [3 marks]** Construct the optimal Huffman tree. Draw the final tree, showing the character and frequency at each leaf node, and the sum of frequencies at each internal node. Show the key intermediate steps of tree construction.

**(b) [1 mark]** Write the resulting binary code for each character. Assume that left branches are labeled with `0` and right branches are labeled with `1` (or vice versa, specify which convention you used).

**(c) [1 mark]** Compute the expected (average) number of bits per character under this Huffman code. How does it compare to the number of bits required per character using a standard fixed-length binary code for 5 characters? (State the percentage of space saved).

---

## Solution

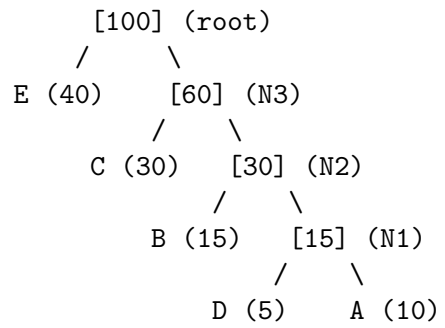
### Part (a) — Tree Construction Steps

1. **Initial Forest:** The priority queue sorted by frequency contains:
   - $D : 5$
   - $A : 10$
   - $B : 15$
   - $C : 30$
   - $E : 40$
2. **Step 1:** Merge the two lowest frequencies, $D(5)$ and $A(10)$.
   - Create parent node $N_1$ with weight $5 + 10 = 15$.

- The tree $N_1$ has children $D$ and $A$.
- Queue now contains: $B : 15$, $N_1 : 15$, $C : 30$, $E : 40$.

3. **Step 2:** Merge the two lowest, $B(15)$ and $N_1(15)$.
   - Create parent node $N_2$ with weight $15 + 15 = 30$.
   - Queue now contains: $C : 30$, $N_2 : 30$, $E : 40$.
4. **Step 3:** Merge the two lowest, $C(30)$ and $N_2(30)$.
   - Create parent node $N_3$ with weight $30 + 30 = 60$.
   - Queue now contains: $E : 40$, $N_3 : 60$.
5. **Step 4:** Merge the remaining two nodes, $E(40)$ and $N_3(60)$.
   - Create the root node with weight $40 + 60 = 100$.

**Final Huffman Tree Structure:**




---

### Part (b) — Binary Prefix Codes

Using the convention **Left Branch = `0`**, **Right Branch = `1`**:

- **E:** `0` (length 1)
- **C:** `10` (length 2)
- **B:** `110` (length 3)
- **D:** `1110` (length 4)
- **A:** `1111` (length 4)

*(Note: If students used Right Branch = `0` and Left Branch = `1`, the codes will be symmetric (e.g. E = `1`, C = `01`, etc.) and are fully correct.)*

---

### Part (c) — Average Code Length & Space Savings

#### 1. Average Bits per Character

$$\begin{aligned}\text{Average Length} &= \sum(\text{Probability} \times \text{Codeword Length}) \\ &= \frac{10}{100}(4) + \frac{15}{100}(3) + \frac{30}{100}(2) + \frac{5}{100}(4) + \frac{40}{100}(1) \\ &= 0.40 + 0.45 + 0.60 + 0.20 + 0.40 \\ &= \mathbf{2.05} \text{ bits per character}\end{aligned}$$

## 2. Comparison to Fixed-Length Coding

- A standard fixed-length binary code representing 5 characters requires $\lceil \log_2 5 \rceil = \mathbf{3}$ bits per character (e.g., `000` to `100`).
- Space saved per character $= 3 - 2.05 = 0.95$ bits.
- Percentage savings:

$$\frac{3 - 2.05}{3} \times 100\% = \frac{0.95}{3} \times 100\% \approx \mathbf{31.67}\%$$